

动态规划优化

台下观众欣喜若狂

Cap1taL

2026年7月

前言

动态规划作为一种分解问题为子问题的算法与思想，其核心正在于问题阶段的划分。改变刻画状态空间的方式，寻找独特的问题结构……

如何找出更好的状态往往是优化的重难点

一般而言，当状态过于繁杂时，我们通常需要跳出现有的状态限制，思考状态是否存在冗余？状态的某一维度是否存在某种性质，使得我们可以保留它的特征信息？而当状态过于简单导致转移出现困难时，我们可以尝试根据转移时遇到的困难（是否有情况无法区分或重复贡献？）去倒推我们所需要的状态划分

有时，相同的转移方程与状态，甚至可以有截然不同的理解方式。动态规划状态的设计是一门精妙的艺术

洛谷 P10299 [CCC 2024 S5] Chocolate Bar Partition

$2 \times N$ 的巧克力，每个小方块有美味度 $T_{i,j}$ 。将巧克力分成若干四连通块，使每个块的平均美味度相同，最大化块数。

$$1 \leq N \leq 2 \times 10^5, 0 \leq T_{i,j} \leq 10^8$$

Solution

- 建模神题
- 平均数太诡异，我们先全部减平均数，变成找和为零的联通块
- 一个朴素的想法是设 $f_{i,j,k}$ ，分别记录末尾两个联通块的和，转移枚举两个新块的归属，维护贡献即可
- 这个状态设计没有前途，我们需要优化状态
- 我们观察到，若两个块和均不为零，我们的转移是固定的，即各分配一个。
- 因此我们希望直到某个块和为 0 时再处理转移
- 设计状态 $f_{i,j} (j \in \{0, 1\})$ 表示处理了前 i

Solution

状态设计优化

- 设计状态 $f_{i,j} (j \in \{0,1\})$ 表示处理了前 i 列，第 j 行的块和不一定为 0
- 考虑转移
 - 新一列的两个都并入 "不一定为 0" 联通块
 - 另一行通过独立出一段和为 0 的块从而合法。当前行延续 "不一定为 0"
 - 另一行通过取一段与之前的非法拼起来从而合法。当前行的部分 "不一定为 0"。需要满足转移条件
 - 检查当前所谓的 "不一定为 0" 联通块是否一定是合法的，如果是则立即结算贡献，从而保证块数最大化

Codeforces 2201C Rigged Bracket Sequence

给定正规括号序列 S （长度为 n ）。选择一个非空子序列将其循环右移一位（第 j 个选中位置得到原第 $j-1$ 个选中位置的字符）。

统计有多少个非空子序列在右移后 S 仍为正规括号序列。

$$2 \leq n \leq 3 \times 10^5, \sum n \leq 3 \times 10^5$$

Solution

- 状态设计动机与性质相关
- 把判断括号序列合法转成判断前缀和非负
- 考虑操作序列中的两个相邻位置 (i, j) , 同时考虑 $i < t < j$
- 那么对于 t 来说, 前方多了操作的最后一个括号, 少了第 i 括号
- 所以显然我们操作的最后一个括号是 $)$ 时才有可能不合法
- 设 f_i 表示前 i , 最后一个操作的是 i 的方案数
- 如果是右括号, 则无任何限制
- 如果是左括号, 枚举 j , 能转移当且仅当 j 是左括号, 或他们中间的位置均不小于 2

Codeforces 2001E2 Deterministic Heap (Hard Version)

大小为 $2^n - 1$ 的完美二叉树（最大堆），初始全 0。每次操作选择一个节点 v ，将根到 v 路径上所有节点 $+1$ 。

若某堆的前两次 pop 操作均确定（每次选择子节点时值不相等），称其为确定性最大堆。求恰好 k 次操作后能得到多少种不同的确定性最大堆。

$2 \leq n \leq 100$, $1 \leq k \leq 500$, $\sum n \leq 100$, $\sum k \leq 500$, p 为质数

Solution

- 考虑两次 pop 操作的形态，发现第二次 pop 形如：先走一段第一次 pop 的操作，再拐弯去别的子树，此后相当于第一次 pop
- 这启示我们类似 "分段" 地 DP
- 考虑第二次 pop 时最为特殊的分叉点 u ，我们不妨假设第一次 pop 去了它的左儿子，第二次却去了右儿子
- 这要求它的左儿子的两个儿子，均比它的右儿子小，这样才会拐弯
- 反之亦然，如果两次 pop 都去了左儿子，则要求左儿子的两个儿子，存在一个比它的右儿子大

Solution

状态设计

- 首先计算 0 次 pop 的情况，只有操作合法的要求
- 设 $f_{i,k}$ 表示子树高为 i ，根节点值是 k 的方案数

$$f_{i,k} = \sum_{x+y \leq k; x \neq y} f_{i-1,x} \times f_{i-1,y}$$

Solution

状态设计

- 再来计算一次 pop。
- 设 $g_{i,k,v}$ 表示树高为 i 根节点为 k ，两个儿子都不大于 v 的方案数

$$g_{i,k,v} = 2 \times \sum_{x+y \leq k; x < y \leq v} f_{i-1,x} \times g_{i-1,y,y}$$

- 其中的系数 2 表示不知道哪一侧更大

Solution

状态设计

- 再来计算两次 pop。
- 设 $h_{i,k,v}$ 表示树高为 i 根节点为 k ，两个儿子的最大值不小于 v 的方案数

$$h_{i,k,v} = 2 \times \sum_{x+y \leq k; x < y; v \leq y} (g_{i-1,y,x-1} \times g_{i-1,x,x} + h_{i-1,y,j+1} f_{i-1,x})$$

- 其中的系数 2 表示不知道哪一侧更大
- 第一项表示分叉向两侧，第二项表示同侧删两次。各自有各自的价值大小要求，和之前的分析保持一致

Solution

状态的设计是为转移服务的

前言

- 均为较为基本且常用的优化手段
- 前缀和并不一定是求和，也有可能是保留最值等信息或者后缀信息，其代表一类预处理的思想
- 单调队列，单调栈优化在做的事情本质和它们的模板一样，这里以单调队列为例，单调栈是类似的

前言

- 朴素的单调队列解决的是滑动窗口的最值问题
- 在DP中，有时决策转移区间会随着某一维状态的变化而变化。一般来说，这类转移可以使用数据结构进行加速，而当转移区间的左右端点均单调移动时，我们可以使用单调队列优化
- 单调队列可以优化的转移形如：

$$dp_i = \max_{j \in [L_i, R_i]} candidate_j + something$$

- 有一些题目的转移并不直接符合上式，却仍可以使用单调队列加速。我们需要对方程和状态做观察变形，这类变形通常较为套路，我们以下面的题目为例

洛谷 B2173 多重背包

n 种物品，背包容量 V 。第 i 种物品体积 w_i 、价值 val_i 、最多 c_i 件。求不超过容量的最大总价值。

$1 \leq n \leq 500, 1 \leq V \leq 1000, 1 \leq c_i \leq 100$

Solution

核心观察是转移有点类似一个一个跳，但是相邻 j 转移点完全没法复用，因此按模 v 分类，一类一类转移，然后可以变成滑动窗口，窗口大小固定为物品数量

洛谷 P16692 染色 Plus

$1 \times N$ 网格， M 种颜料，每种在 $[L_i, R_i]$ 内可用，涂一格花费 C_i 。每格需涂一种可用颜料，且任意连续 K 格颜色不能全相同。求最小总代价，或输出 -1 。

$2 \leq N, M \leq 2 \times 10^5$, $2 \leq K \leq N$, $1 \leq C_i \leq 10^9$

Solution

- 首先观察到，一个位置最多使用最优三个颜色。首先用一点数据结构技巧处理出每个颜色可能被使用的区间
- 题目限制等价于单颜色段不能超过 $K - 1$
- 我们设 $dp_{i,c}$ 表示前 i 且最后涂的颜色为 c 。第二维不是完整的 $O(n)$ 数组，而总是存储前三个最优颜色，当然，对于不同的 i 最优的颜色也不一样。

$$dp_{i,c} = \min_{\substack{j \in [i-(K-1), i], \\ [j+1, i] \subseteq \{c \text{ 可行区间}\}, c' \neq c}} (dp_{j,c'} + C(i-j))$$

- 看起来非常滑动窗口，但颜色的限制让我们非常难以处理

Solution

- 设 $g_{i,c} = \min_{c' \neq c} (f_{i-1,c'})$
- 依旧只对 i 有用的三个 c 进行处理——可以 $O(1)$ 计算。
- 因此转移变为

$$dp_{i,c} = \min_{\substack{j \in [i-(K-1), i], \\ [j+1, i] \subseteq \{c \text{ 可行区间}\}}} (g_{j+1,c} + C(i-j))$$

- 颜色独立，对每个颜色维护单调序列，可以解决滑动窗口转移

前言

若我们的 DP 转移形如

$$dp_i = \min\{dp_j + a(i) \times b(j) + c(i) + d(j)\}$$

则我们可以整理为

$$dp_i - c(i) = \min\{(dp_j + d(j)) - (-a(i) \times b(j))\}$$

此时若我们看作一个点 $(b(j), dp_j + d(j))$ ，会发现我们要做的，实际上是选取一些这样的点 j ，用一条斜率为 $-a(i)$ 的点穿过，求截距的最小值

自然地，我们会尝试维护凸包

斜率优化的“凸”是由转移的数学形式而来的，某种意义上，可以说与题目无关（对比 wqs 二分的“凸”）

斜率优化四种情形汇总

$x_j \setminus k_i$	k_i 单调	k_i 不单调
x_j 单调	单调队列	凸包二分
x_j 不单调	CDQ / 李超树	CDQ / 李超树

我们来看具体例题

洛谷 P3195 [HNOI2008] 玩具装箱

n 件玩具排成一排，第 i 件长度为 C_i 。将连续若干件装入一容器，若装 $[i, j]$ ，则容器长度 $x = j - i + \sum_{k=i}^j C_k$ ，费用 $(x - L)^2$ 。求最小总费用。

$1 \leq n \leq 5 \times 10^4$, $1 \leq L \leq 10^7$, $1 \leq C_i \leq 10^7$

Solution

- 转移方程是显然的
- 拆开平方，会存在 $w(i)$ 与 $w(j)$ 的交叉项
- 正是这个交叉二次项指引我们使用斜率优化
- 具体结合 DP转移推导
- 一大关键是，本题中玩具价值总为正，这保证了我们的横坐标为升序，因此可以直接维护凸包

洛谷 P5504 [JSOI2011] 柠檬

n 个贝壳排成一排，每个有大小 s_i 。每次取一段连续贝壳，选一个大小 s_0 ，若该段中大小为 s_0 的有 t 个，则得到 $s_0 t^2$ 只柠檬。求将贝壳取完能得到的最大柠檬数。

$1 \leq n \leq 10^5, 1 \leq s_i \leq 10^4$

Solution

- 一个显然的观察：我们选取的区间两端贝壳大小必定相同，且是我们指定的颜色，否则不选肯定更优
- 设 cnt_i 代表前缀意义上 a_i 出现了多少次，有转移

$$dp_i = \max_{a_j = a_i} (dp_{j-1} + a_i(cnt_i - cnt_j + 1)^2)$$

- 观察到平方，知道拆开后会出现交叉项。整理

$$dp_i - a_i(cnt_i + 1)^2 = \max_{a_j = a_i} (dp_{j-1} + a_i cnt_j^2 - 2a_i cnt_j (cnt_i + 1))$$

- 同时存在两种交叉项，似乎不好处理

Solution

$$dp_i - a_i(cnt_i + 1)^2 = \max_{a_j = a_i} (dp_{j-1} + a_j cnt_j^2 - 2a_i cnt_j (cnt_i + 1))$$

- 注意到我们的性质保证了 $a_j = a_i$ ，于是只需替换

$$dp_i - a_i(cnt_i + 1)^2 = \max_{a_j = a_i} (dp_{j-1} + a_j cnt_j^2 - 2a_i cnt_j (cnt_i + 1))$$

- 再整理

$$dp_i - a_i(cnt_i + 1)^2 = \max_{a_j = a_i} (dp_{j-1} + a_j cnt_j^2 - 2a_i (cnt_i + 1) cnt_j)$$

- “横坐标”是 (cnt_j) 单调，于是直接维护凸包并二分即可

前言

- CDQ分治还可以用来优化一类 1D/1D 的动态规划
- 我们关注到，用 dp_j 更新 dp_i 也类似一种“点对”关系
- CDQ 分治可以改变一类转移的顺序，同时也可以配合斜率优化使用
- 我们结合题目来讲解

洛谷 P2487 [SDOI2011] 拦截导弹

导弹有高度 h_i 和速度 v_i ，拦截系统要求后一发炮弹的高度和速度均不高于前一发。求最多能拦截的导弹数量，以及每枚导弹被拦截的概率（所有最优方案等概率选取）。

$$1 \leq n \leq 5 \times 10^4, 1 \leq h_i, v_i \leq 10^9$$

Solution

- 一个二维LDS问题
- 转移的时候有三维偏序限制。暴力做法可以顺着扫描线去掉一维，再使用树套树等数据结构处理二位偏序，不过这非常坏
- 我们使用CDQ分治来解决这类问题
- 具体地，我们使用 $CDQ(l,r)$ 来计算出内部所有的 dp_i ，每次先向左递归，再用左半更新右半，最后向右递归
- 在最关键的用左侧更新右侧部分，我们可以把左右都按照 a 排序，再把 b 插入一维数据结构即可
- 注意按 a 排序无法使用归并，因为右侧部分还没有向下递归——我们不能更改他们之间的顺序，否则转移顺序就会被破坏

辅助斜率优化

$$dp_i - c(i) = \min\{(dp_j + d(j)) - (-a(i) \times b(j))\}$$

若点 $(b(j), dp_j + d(j))$ 的插入横坐标不是升序的，则我们无法简单的通过弹栈来维护凸包

暴力做法是用平衡树维护凸包并在平衡树上二分，不过能否用更简单的方法呢？

辅助斜率优化

$$dp_i - c(i) = \min\{(dp_j + d(j)) - (-a(i) \times b(j))\}$$

我们使用 CDQ 分治，在递归左侧之后，将左侧按照横坐标升序，右侧的“待更新”按照“斜率”升序

之后我们扫描左侧，用栈维护出凸包，再扫描右侧回答询问即可。复杂度瓶颈是排序，扫描均为线性，也无需进行二分

洛谷 P4655 [CEOI 2017] Building Bridges

n 根柱子，高度 h_i ，拆除代价 w_i 。可在 i, j 间建桥，代价 $(h_i - h_j)^2$ 。用桥连接第 1 根和第 n 根柱子（桥梁不相交），求最小总代价（建桥+拆除）。

$2 \leq n \leq 10^5$, $0 \leq h_i \leq 10^6$, $|w_i| \leq 10^6$

- 转移方程显然

$$dp_i = \min (dp_j + (h_i - h_j)^2 + s_{i-1} - s_j)$$

- 拆开并整理

$$dp_i - h_i^2 - s_{i-1} = \min (h_j^2 + dp_j - s_j - 2h_i h_j)$$

- 套用斜率优化，我们发现“横坐标”是 h_j ，很不幸，它不是单调的
- 因此使用 CDQ 分治，每次把左侧对 h 排序后正常维护凸包，再用来更新右侧的 dp_i 即可

前言

类似wqs二分，这一部分的讲解，不会使用四边形不等式对决策单调性做出证明
我们只讲解它的部分常用情形，以及怎么在做题时使用

前言

- 我们常常会见到这样的转移

$$dp_{i,j} = \min_k w(i,j,k)$$

- 其中的 $w(i,j,k)$ 可能类似 $dp_{i-1,k} + something(i,j,k)$ 的形式，总之是一个与 i,j 有关的函数
- 决策单调性，顾名思义，表示若 $j_2 > j_1$ ，则对他们各自最优秀的 w 满足 $k_2 > k_1$ ，即随着 j 增大，最优秀的 k 也在增大
- 如果 $w(i,j,k)$ 可以 $O(1)$ 计算，或 $w(i,j,k)$ 可以 $O(1)$ 地转移到 $O(i,j \pm 1,k)$ 和 $O(i,j,k \pm 1)$ （类似莫队），则我们可以使用分治法加速

前言

- 具体地，我们使用 $\text{solve}(l,r,L,R)$ 来计算出所有 $dp_{i,l} dp_{i,r}$ ，保证他们的最优转移区间是 $[L, R]$
- 我们计算 $mid = \frac{l+r}{2}$ ，然后暴力计算 $dp_{i,mid}$ 并得到它的最优转移点 p ，递归到 $\text{solve}(l,mid-1,L,p)$ 和 $\text{solve}(mid+1,r,p,R)$ 。复杂度显然为 $O(n \log n)$
- 如果 w 不能快速计算只能 $O(1)$ 改变至相邻区间，我们只需在暴力计算 $dp_{i,mid}$ 时暴力移动 w 复杂度仍为 $O(n \log n)$
- 同递归树内移动次数 $O(n)$ ，总共 $O(\log)$ 层，因此均摊 $O(1)$
- 具体题目

Codeforces 868F Yet Another Minimization Problem

将序列 a 分成 k 个子段，每个子段的费用是该段内相同元素的对数。求所有子段费用之和的最小值。

$$2 \leq n \leq 10^5, 2 \leq k \leq \min(n, 20), 1 \leq a_i \leq n$$

Solution

- DP式子显然
- 发现有决策单调性
- 问题在于 w ，发现只需要维护桶，那么移动是简单的
- 复杂度 $kn \log n$

Thanks!